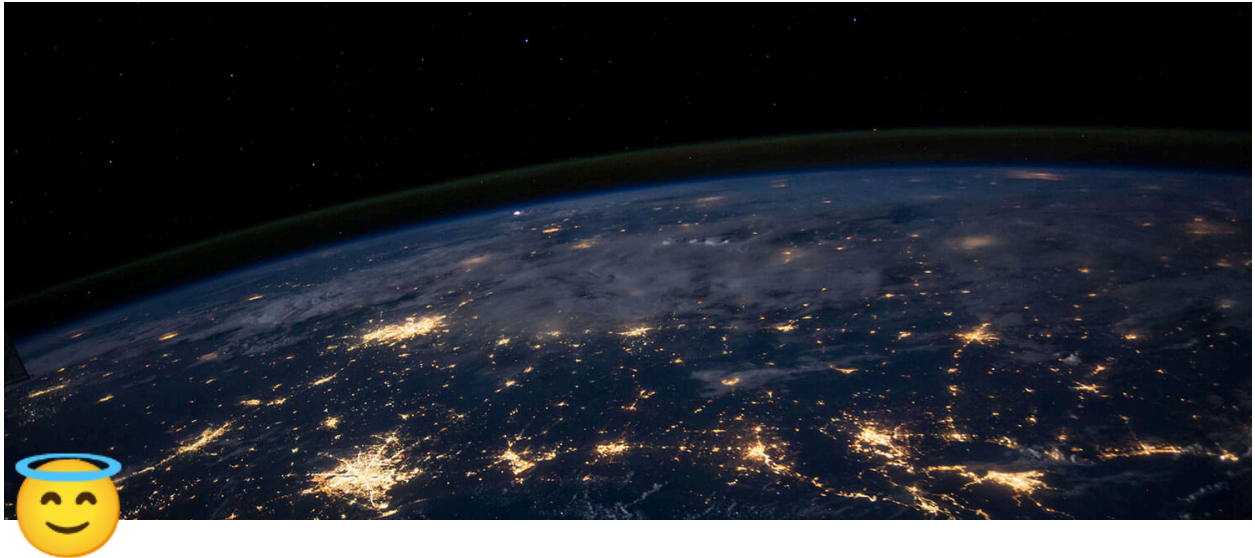




Numpy notes

1. NumPy Introduction

- NumPy is a Python library used for working with arrays.
- NumPy còn có 1 số hàm làm việc trong môi trường đại số tuyến tính, biến đổi fourier và ma trận.
- NumPy aims to provide an array object that is up to 50x faster than traditional Python lists.
- The array object in NumPy is called `ndarray`, it provides a lot of supporting functions that make working with `ndarray` very easy.
- NumPy arrays are stored at one continuous place in memory unlike lists, so processes can access and manipulate them very efficiently.

2. NumPy Getting Started

- NumPy is usually imported under the `np` alias.
- Create an alias with the `as` keyword while importing:

```
import numpy as np
```

3. NumPy Creating Arrays

- NumPy is used to work with arrays. The array object in NumPy is called `ndarray`.
- We can create a NumPy `ndarray` object by using the `array()` function.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print(arr)
print(type(arr))
```

- To create an `ndarray`, we can pass a list, tuple or any array-like object into the `array()` method, and it will be converted into an `ndarray`.

a. *Dimensions in Arrays*

- Số chiều của 1 mảng phụ thuộc vào độ sâu của mảng.
- Mảng 1 chiều bình thường và mảng 2 chiều (đã học).
- Một mảng chứa các mảng 2 chiều (ma trận) làm phần tử của nó thì được gọi là mảng 3 chiều (3-D array). Chúng thường được sử dụng để biểu diễn một Tensor bậc 3.

```
import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])
print(arr)
```

- NumPy Arrays provides the `ndim` attribute that returns an integer that tells us how many dimensions the array have.
- An array can have any number of dimensions.
- When the array is created, you can define the number of dimensions by using the `ndmin` argument.

```
#Create an array with 5 dimensions and verify that it has 5 dimensions
```

```
import numpy as np
arr = np.array([1, 2, 3, 4], ndmin=5)
print(arr)
print('number of dimensions :', arr.ndim)
```

4. NumPy Array Indexing

- To access elements from 2-D arrays we can use comma separated integers representing the dimension and the index of the element.

```
import numpy as np
arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])
print('2nd element on 1st row: ', arr[0, 1])
```

- To access elements from 3-D arrays we can use comma separated integers representing the dimensions and the index of the element.

```
import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
print(arr[0, 1, 2])
```

5. NumPy Array Slicing

- Slicing in python means taking elements from one given index to another given index.
- **Note:** The result *includes* the start index, but *excludes* the end index.
- **Slicing 2-D Arrays**
 - From the second element, slice elements from index 1 to index 4 (not included):

```
import numpy as np
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
print(arr[1, 1:4]) #[7,8,9]
```

- From both elements, return index 2:

```
import numpy as np
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
print(arr[0:2, 2]) #[3,8]
```

- From both elements, slice index 1 to index 4 (not included), this will return a 2-D array:

```
import numpy as np
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
print(arr[0:2, 1:4])
...
[[2 3 4]
 [7 8 9]]
...
```

6. NumPy Data Types

- NumPy has some extra data types, and refer to data types with one character, like `i` for integers, `u` for unsigned integers etc.
- Below is a list of all data types in NumPy and the characters used to represent them.
 - `i` - integer
 - `b` - boolean
 - `u` - unsigned integer
 - `f` - float
 - `c` - complex float
 - `m` - timedelta
 - `M` - datetime
 - `O` - object
 - `S` - string

- `U` - unicode string
- `V` - fixed chunk of memory for other type (void)
- The NumPy array object has a property called `dtype` that returns the data type of the array. (thuộc tính not phương thức)
- We use the `array()` function to create arrays, this function can take an optional argument: `dtype` that allows us to define the expected data type of the array elements.
 - Create an array with data type string:

```
import numpy as np
arr = np.array([1, 2, 3, 4], dtype='S')
print(arr)
print(arr.dtype)
```

- For `i`, `u`, `f`, `S` and `U` we can define size as well.
 - Create an array with data type 4 bytes integer:

```
import numpy as np
arr = np.array([1, 2, 3, 4], dtype='i4')
print(arr)
print(arr.dtype)
```

- If a type is given in which elements can't be casted then NumPy will raise a `ValueError`.
- The `astype()` function creates a copy of the array, and allows you to specify the data type as a parameter.
 - Change data type from float to integer by using `'i'` as parameter value:

```
import numpy as np
arr = np.array([1.1, 2.1, 3.1])
newarr = arr.astype('i')
```

```
print(newarr)
print(newarr.dtype)
...
[1 2 3]
int32
...
```

7. NumPy Array Copy vs View

- The main difference between a copy and a view of an array is that the copy is a new array, and the view is just a view of the original array.
 - Make a copy, change the original array, and display both arrays:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
x = arr.copy()
arr[0] = 42
print(arr) #[42  2  3  4  5]
print(x) #[1 2 3 4 5]
```

- Make a view, change the original array, and display both arrays:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
x = arr.view()
arr[0] = 42
print(arr) #[42  2  3  4  5]
print(x) #[42  2  3  4  5]
```

- The copy SHOULD NOT be affected by the changes made to the original array.
- The view SHOULD be affected by the changes made to the original array. (và ngược lại)
- Every NumPy array has the attribute `base` that returns `None` if the array owns the data.

- Otherwise, the `base` attribute refers to the original object.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
x = arr.copy()
y = arr.view()
print(x.base) #None
print(y.base) #[1 2 3 4 5]
```

- The copy returns `None`.
- The view returns the original array.

8. NumPy Array Shape (Hình dáng/Kích thước)

- The shape of an array is the number of elements in each dimension.
- NumPy arrays have an attribute called `shape` that returns a tuple with each index having the number of corresponding elements.

```
import numpy as np
# Tạo một ma trận 2 hàng, 3 cột
A = np.array([[1, 2, 3],
              [4, 5, 6]])
print(A.shape)
# Kết quả: (2, 3) -> 2 hàng, 3 cột
```

- Integers at every index tells about the number of elements the corresponding dimension has.

```
import numpy as np
arr = np.array([1, 2, 3, 4], ndmin=5)
print(arr) #[[[[1 2 3 4]]]]
print('shape of array:', arr.shape) #shape of array:
(1, 1, 1, 1, 4)
```

9. NumPy Array Reshaping

- Reshaping means changing the shape of an array.

a. **Reshape From 1-D to 2-D**

- Convert the following 1-D array with 12 elements into a 2-D array. The outermost dimension will have 4 arrays, each with 3 elements:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
newarr = arr.reshape(4, 3)
print(newarr)
...
```

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
...
```

b. **Reshape From 1-D to 3-D**

- Convert the following 1-D array with 12 elements into a 3-D array. The outermost dimension will have 2 arrays that contains 3 arrays, each with 2 elements:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
newarr = arr.reshape(2, 3, 2)
print(newarr)
#[[[1,2],[3,4],[5,6]],[[7,8],[9,10],[11,12]]]
```

c. **Can We Reshape Into any Shape?**

→ Yes, as long as the elements required for reshaping are equal in both shapes.

- We can reshape an 8 elements 1D array into 4 elements in 2 rows 2D array but we cannot reshape it into a 3 elements 3 rows 2D array as that would require $3 \times 3 = 9$ elements.
- reshape chỉ là 1 view không phải là 1 copy.

d. **Unknown Dimension**

- You are allowed to have one "unknown" dimension. Meaning that you do not have to specify an exact number for one of the dimensions in the reshape method.
- Pass `-1` as the value, and NumPy will calculate this number for you.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
newarr = arr.reshape(2, 2, -1)
print(newarr)
```

- **Note:** We can not pass `-1` to more than one dimension.

e. **Flattening the arrays**

- Flattening array means converting a multidimensional array into a 1D array. (trải phẳng mảng nhiều chiều)
- We can use `reshape(-1)` to do this.
 - Convert the array into a 1D array:

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
newarr = arr.reshape(-1)
print(newarr) #[1 2 3 4 5 6]
```

10. **NumPy Array Iterating**

a. **Iterating Arrays**

- Iterating means going through elements one by one.

- As we deal with multi-dimensional arrays in numpy, we can do this using basic `for` loop of python.

b. *Iterating 2-D Arrays*

- In a 2-D array it will go through all the rows.

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
for x in arr:
    print(x)
...
[1 2 3]
[4 5 6]
...
```

- If we iterate on a n -D array it will go through $n-1$ th dimension one by one.
- Để trả về các giá trị thực tế, các giá trị vô hướng, chúng ta phải lặp lại các mảng theo từng chiều.

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
for x in arr:
    for y in x:
        print(y)
```

c. *Iterating 3-D Arrays*

- In a 3-D array it will go through all the 2-D arrays.
- To return the actual values, the scalars, we have to iterate the arrays in each dimension.

```
import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
```

```
for x in arr:
    for y in x:
        for z in y:
            print(z)
```

d. *Iterating Arrays Using `nditer()`*

- The function `nditer()` is a helping function that can be used from very basic to very advanced iterations.
- In basic `for` loops, iterating through each scalar of an array we need to use n `for` loops which can be difficult to write for arrays with very high dimensionality.

```
import numpy as np
arr = np.array([[[[1, 2], [3, 4]], [[5, 6], [7, 8]]]])
for x in np.nditer(arr):
    print(x) #1 2 3 4 5 6 7 8
```

e. *Iterating Array With Different Data Types*

- NumPy quản lý bộ nhớ rất chặt. Nếu muốn ép kiểu dữ liệu tạm thời trong lúc chạy vòng lặp, bạn phải xin phép cấp thêm RAM (buffer) để nó có chỗ xử lý.
- NumPy yêu cầu mảng phải liên tục và đồng nhất. Mỗi kiểu dữ liệu chiếm một dung lượng khác nhau trong RAM (ví dụ: số nguyên `int32` chiếm 4 bytes, số thực `float64` chiếm 8 bytes).
 - Nếu bạn đang ở ô nhớ 4 bytes, bạn không thể nhét một con số 8 bytes vào đó được. Nó sẽ tràn sang ô của phần tử bên cạnh và làm hỏng toàn bộ cấu trúc mảng.
 - Do đó, NumPy **từ chối** việc chuyển đổi kiểu dữ liệu trực tiếp ngay trên mảng gốc.
- Vì không thể ghi đè trực tiếp lên mảng gốc, NumPy cần một không gian làm việc tạm thời.

- `flags=['buffered']` : NumPy mặc định vô cùng tiết kiệm RAM và không tự động tạo ra tở nháp này. Bạn phải khai báo lệnh này để cấp quyền.

```
import numpy as np
# Mảng gốc chứa các số nguyên (int)
arr = np.array([1, 2, 3])
# Vòng lặp báo lỗi nếu KHÔNG CÓ flags=['buffered']
# for x in np.nditer(arr, op_dtypes=['S']):
#     print(x) # -> TypeError!

# Vòng lặp chuẩn xác
for x in np.nditer(arr, flags=['buffered'], op_dtypes=['S']):
    print(x)
    # Output sẽ là b'1', b'2', b'3' (chữ 'b' biểu thị
    # dạng chuỗi byte trong Python)
```

f. *Iterating With Different Step Size*

- Iterate through every scalar element of the 2D array skipping 1 element:

```
import numpy as np
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
for x in np.nditer(arr[:, ::2]):
    print(x) #1 3 5 7
```

- Hàm `np.ndenumerate()` cho phép bạn duyệt qua từng phần tử của một mảng NumPy nhiều chiều. Tại mỗi bước lặp, nó sẽ trả về cho bạn 2 thứ:
 1. **Tọa độ (Index):** Được gói trong một Tuple (ví dụ: `(0, 1)` nghĩa là hàng 0, cột 1).
 2. **Giá trị (Value):** Con số nằm chính xác tại tọa độ đó.

```
import numpy as np
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
for idx, x in np.ndenumerate(arr):
    print(idx, x)
...
(0, 0) 1
(0, 1) 2
(0, 2) 3
(0, 3) 4
(1, 0) 5
(1, 1) 6
(1, 2) 7
(1, 3) 8
...
```

11. NumPy Joining Array

- Joining means putting contents of two or more arrays in a single array.
- Trong NumPy, mảng có bao nhiêu chiều (dimension) thì sẽ có bấy nhiêu trục (axis). Trục đơn giản là hướng mà bạn muốn thực hiện hành động.
 - Ví dụ: Tưởng tượng một mảng 2 chiều (Ma trận) giống như một bảng tính Excel: (Không đúng với stack!)
 - **Axis 0 (Trục 0 - Chiều dọc):** Nó chạy từ trên xuống dưới, xuyên qua các **hàng**.
 - **Axis 1 (Trục 1 - Chiều ngang):** Nó chạy từ trái sang phải, xuyên qua các **cột**.
- Trường hợp 1: Gộp theo `axis=0` (Nối dọc - Mặc định)
 - NumPy đi theo hướng từ trên xuống dưới. Nó sẽ lấy ma trận B **đặt bồi thêm xuống phía dưới** ma trận A. → *Kết quả*: Số hàng tăng lên, số cột giữ nguyên.

```
import numpy as np
A = np.array([[1, 2],
```

```

        [3, 4]])
B = np.array([[5, 6],
              [7, 8]])
gop_truc_0 = np.concatenate((A, B), axis=0)
print(gop_truc_0)
# Kết quả:
# [[1 2]
#  [3 4]
#  [5 6]
#  [7 8]]

```

- Trường hợp 2: Gộp theo `axis=1` (Nối ngang)
 - NumPy đi theo hướng từ trái qua phải. Nó sẽ lấy ma trận B **đặt sang bên phải** của ma trận A. → *Kết quả*: Số hàng giữ nguyên, số cột tăng lên.

```

gop_truc_1 = np.concatenate((A, B), axis=1)
print(gop_truc_1)
# Kết quả:
# [[1 2 5 6]
#  [3 4 7 8]]

```

a. *Joining Arrays Using Stack Functions*

- Sự khác biệt giữa `concatenate` và `stack`
 - `concatenate()` : Chắp vá các mảng lại với nhau trong **cùng một không gian (số chiều không đổi)**. Gộp hai ma trận 2D vẫn ra một ma trận 2D to hơn.
 - `stack()` : Không chắp vá, mà nó **xếp chồng** các mảng lên nhau để tạo ra một **chiều không gian mới**. Gộp hai ma trận 2D bằng `stack` sẽ đẻ ra một khối Tensor 3D (giống như lấy 2 tờ giấy rồi xếp lên nhau thành một xấp giấy).
- Axis khi dùng stack ngược lại với `concatenate()` (chắc là thế ?)

- `axis=0` (Trục Hàng) → "Hãy biến các mảng của tôi thành các HÀNG."
- `axis=1` (Trục Cột) → "Hãy biến các mảng của tôi thành các CỘT".
- `vstack()` (Vertical Stack - Xếp dọc): Xếp các mảng chồng lên nhau từ trên xuống dưới (trở thành các hàng).
- `hstack()` (Horizontal Stack - Xếp ngang): Nối các mảng sát cạnh nhau từ trái qua phải.
- `dstack()` (Depth Stack - Xếp theo chiều sâu): Nó xếp chồng các mảng lên nhau theo trục không gian thứ 3 (trục Z), giống như xếp các tấm kính trong suốt lên nhau.

12. NumPy Splitting Array

- Splitting is reverse operation of Joining.
- Joining merges multiple arrays into one and Splitting breaks one array into multiple.
- We use `array_split()` for splitting arrays, we pass it the array we want to split and the number of splits.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6])
newarr = np.array_split(arr, 3)
print(newarr)
#[array([1, 2]), array([3, 4]), array([5, 6])]
```

- **Note:** The return value is a list containing three arrays.

- If the array has less elements than required, it will adjust from the end accordingly.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6])
newarr = np.array_split(arr, 4)
print(newarr)
```

```
#[array([1, 2]), array([3, 4]), array([5]), array([6])]
```

- **Note:** We also have the method `split()` available but it will not adjust the elements when elements are less in source array for splitting like in example above, `array_split()` worked properly but `split()` would fail.
- Vì phương thức `array_split()` trả về 1 danh sách các mảng nên ta có thể truy cập từng mảng bằng index như bình thường.

a. **Splitting 2-D Arrays**

- Use the `array_split()` method, pass in the array you want to split and the number of splits you want to do.

```
import numpy as np
arr = np.array([[1, 2], [3, 4], [5, 6], [7, 8], [9, 10], [11, 12]])
newarr = np.array_split(arr, 3)
print(newarr)
...
[array([[1, 2],
        [3, 4]]), array([[5, 6],
        [7, 8]]), array([[ 9, 10],
        [11, 12]])]
...
```

- In addition, you can specify which axis you want to do the split around.
- An alternate solution is using `hsplit()` opposite of `hstack()`.
- **Note:** Similar alternates to `vstack()` and `dstack()` are available as `vsplit()` and `dsplit()`.

13. **NumPy Searching Arrays**

- You can search an array for a certain value, and return the indexes that get a match.

- To search an array, use the `where()` method.

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 4, 4])
x = np.where(arr == 4)
print(x)
```

- The example above will return a tuple: `(array([3, 5, 6]),)`

a. **Search Sorted**

- There is a method called `searchsorted()` which performs a binary search in the array, and returns the index where the specified value would be inserted to maintain the search order.
- The `searchsorted()` method is assumed to be used on sorted arrays. (chỉ dùng cho mảng đã sắp xếp)

```
import numpy as np
arr = np.array([6, 7, 8, 9])
x = np.searchsorted(arr, 7)
print(x) #1
```

- Example explained: The number 7 should be inserted on index 1 to remain the sort order.
- The method starts the search from the left and returns the first index where the number 7 is no longer larger than the next value.
- By default the left most index is returned, but we can give `side='right'` to return the right most index instead.

b. **Multiple Values**

- To search for more than one value, use an array with the specified values.

```
import numpy as np
arr = np.array([1, 3, 5, 7])
```

```
x = np.searchsorted(arr, [2, 4, 6])
print(x) #[1 2 3]
```

- The return value is an array: `[1 2 3]` containing the three indexes where 2, 4, 6 would be inserted in the original array to maintain the order.

14. NumPy Sorting Arrays

- The NumPy ndarray object has a function called `sort()`, that will sort a specified array.

```
import numpy as np
arr = np.array([3, 2, 0, 1])
print(np.sort(arr)) #[0 1 2 3]
```

- **Note:** This method returns a copy of the array, leaving the original array unchanged.
- You can also sort arrays of strings, or any other data type.
- If you use the `sort()` method on a 2-D array, both arrays will be sorted:

```
import numpy as np
arr = np.array([[3, 2, 4], [5, 0, 1]])
print(np.sort(arr))
...
[[2 3 4]
 [0 1 5]]
...
```

15. NumPy Filter Array

- Getting some elements out of an existing array and creating a new array out of them is called *filtering*.
- In NumPy, you filter an array using a *boolean index list*.
 - A *boolean index list* is a list of booleans corresponding to indexes in the array.

- If the value at an index is `True` that element is contained in the filtered array, if the value at that index is `False` that element is excluded from the filtered array.

```
import numpy as np
arr = np.array([41, 42, 43, 44])
x = [True, False, True, False]
newarr = arr[x]
print(newarr) #[41, 43]
```

- The new array contains only the values where the filter array had the value `True`, in this case, index 0 and 2.

a. **Creating the Filter Array**

```
import numpy as np
arr = np.array([41, 42, 43, 44])
# Create an empty list
filter_arr = []
# go through each element in arr
for element in arr:
    # if the element is higher than 42, set the value to
    # True, otherwise False:
    if element > 42:
        filter_arr.append(True)
    else:
        filter_arr.append(False)
newarr = arr[filter_arr]
print(filter_arr) #[False, False, True, True]
print(newarr) #[43 44]
```

b. **Creating Filter Directly From Array**

```
import numpy as np
arr = np.array([41, 42, 43, 44])
filter_arr = arr > 42
```

```
newarr = arr[filter_arr]
print(filter_arr) #[False, False, True, True]
print(newarr) #[43 44]
```